```
/// ■■■■■V3.0■
class AppConstants {
  AppConstants._();

  static const String appName = '■■■■■';
  static const String slogan = '■■■■■■■■■■■■■■■■■';
  static const String watermark = '■■■■■■■ App';
  static const String appVersion = '3.0.0';

  // V3.0 ■■■■
  static const int maxBookTitle = 100;
  static const int maxAuthorName = 50;
  static const int maxFeeling = 200;
  static const int maxCustomMessage = 200;

  // V2.0 ■■■■
  static const int maxTitleLength = maxBookTitle;
  static const int maxAuthorLength = maxAuthorName;
  static const int maxNotesLength = maxFeeling;

  // ■■■■■
  static const int freeVersionLimit = 5;
  static const int maxReadingCardShow = 2;
  static const int softLimitWarning = 3;

  // Pro ■
  static const double proPrice = 12.0;
  static const String proProductId = 'pro_version';

  // ■■■■
  static const int defaultYearlyGoal = 0; // V3.4: ■■■■0■■■■■■■■■■■■■
  static const String defaultReminderTime = '21:00';

  // V3.0 ■■■■■
  static const int shareImageWidth = 360;
  static const int shareImageHeight = 640;
  static const double shareScale = 3.0;

  // V3.0 ■■■■■
  static const double splashMinDuration = 1.5;
  static const double splashMaxDuration = 3.0;

  // ■■■■■■■■■■■
  static const double homeProgressDiameter = 240.0;
  static const double homeProgressStrokeWidth = 16.0;

  // V2.0 ■■
  static const int splashDurationMs = 3000;

  // ■■■■
```

```
  static const int progressAnimationMs = 500;
  static const int successAnimationMs = 1500;

  // ■■■
  static const String dbName = 'reading_progress.db';
  static const int dbVersion = 3; // V3.0

  // ■■■■■■■■■■■■
  static const String privacyLegalText = '■■■■■\n\n■■■■■■■■■■■■App■■■■■■■■■ App ■■■■■■■■■■■■■■■■\n\n1. ■■...

  static const String privacyPolicyText = '■■■■■\n\n■■■■■■■■■■■■■■■■■■■\n\n1. ■■■■■■■■■ App ■■■■■■■■■■■■■■■.

  static const String privacyDataCollectionText = '■■■■■■■■■\n\n■■■■■■■■■■■■■■■■■■■■■■\n\n■■■■■■■■■■■■■■■■.

  static const String privacyThirdPartyText = '■■■■■■■■■■\n\n■ App ■■■■■■■ SDK ■■■\n\n1. Flutter SDK — ...

  static const String privacyOpenSourceText = '■■■■■\n\n■ App ■■ Flutter ■■■■■■■■■■■■■■■■■■■■■\n\n• Flutt...

  // ■■■■■
  static const String contactEmail = 'feedback@reading-progress.app';
}


/// ■■■■■■■
class AssetPaths {
  AssetPaths._();

  // ■■■
  static const String icAdd = 'assets/icons/ic_add.svg';
  static const String icBooks = 'assets/icons/ic_books.svg';
  static const String icCalendar = 'assets/icons/ic_calendar.svg';
  static const String icAuthor = 'assets/icons/ic_author.svg';
  static const String icRating = 'assets/icons/ic_rating.svg';
  static const String icFire = 'assets/icons/ic_fire.svg';
  static const String icLaunchLogo = 'assets/icons/ic_launch_logo.svg';

  // ■■■
  static const String logo = 'assets/images/logo.png';
  static const String emptyBooks = 'assets/images/empty_books.png';
  static const String proFeatures = 'assets/images/pro_features.png';
}


import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

/// ■■■■■■■■■■— V3.0
class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;
```

```
static const String _dbName = 'reading_progress.db';
static const int _dbVersion = 5; // V3.5: version 5 (fix: checkin_details in _onCreate)

DatabaseHelper._init();

Future<Database> get database async {
  if (_database != null) return _database!;
  _database = await _initDatabase();
  return _database!;
}

Future<Database> _initDatabase() async {
  final dbPath = await getDatabasesPath();
  final path = join(dbPath, _dbName);

  return await openDatabase(
    path,
    version: _dbVersion,
    onCreate: _onCreate,
    onUpgrade: _onUpgrade,
  );
}

Future<void> _onCreate(Database db, int version) async {
  // V3.0: books ■■■■■ wish/reading/done + read_count■
  await db.execute('''
    CREATE TABLE books (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      author TEXT NOT NULL DEFAULT '',
      cover_path TEXT,
      rating INTEGER,
      notes TEXT,
      read_date DATE,
      start_date TEXT NOT NULL,
      status TEXT DEFAULT 'wish' CHECK(status IN ('wish', 'reading', 'done', 'abandoned')),
      abandoned_at DATETIME,
      finished_at DATETIME,
      read_count INTEGER NOT NULL DEFAULT 1,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )
  ''');

  // V3.0: year_goals ■■■■■■■■■■■
  await db.execute('''
    CREATE TABLE year_goals (
      year INTEGER PRIMARY KEY,
      target INTEGER NOT NULL DEFAULT 0,
      is_set_by_user INTEGER NOT NULL DEFAULT 0
```

```
    )
  ''');

  // settings ■
  await db.execute('''
    CREATE TABLE settings (
      key TEXT PRIMARY KEY,
      value TEXT
    )
  ''');

  // ■■
  await db.execute('CREATE INDEX idx_status ON books(status)');
  await db.execute('CREATE INDEX idx_read_date ON books(read_date)');
  await db.execute('CREATE INDEX idx_start_date ON books(start_date)');
  await db.execute('CREATE INDEX idx_finish_date ON books(finished_at)');
  await db.execute('CREATE INDEX idx_author ON books(author)');

  // ■■■■
  await db.execute("INSERT OR IGNORE INTO settings VALUES ('yearly_goal', '0')");
  await db.execute("INSERT OR IGNORE INTO settings VALUES ('theme', 'light')");
  await db.execute("INSERT OR IGNORE INTO settings VALUES ('daily_reminder', 'false')");
  await db.execute("INSERT OR IGNORE INTO settings VALUES ('reminder_time', '21:00')");
  await db.execute("INSERT OR IGNORE INTO settings VALUES ('pro_purchased', 'false')");
  await db.execute("INSERT OR IGNORE INTO settings VALUES ('backup_enabled', 'false')");

  // ■■■■■■■■■■
  final year = DateTime.now().year;
  await db.execute(
      "INSERT OR IGNORE INTO year_goals (year, target, is_set_by_user) VALUES ($year, 0, 0)");

  // V3.5: checkin_details ■■■■■■■■■■■■■■
  await db.execute('''
    CREATE TABLE IF NOT EXISTS checkin_details (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      book_id INTEGER NOT NULL,
      checkin_date TEXT NOT NULL,
      duration_min INTEGER,
      note TEXT,
      created_at TEXT DEFAULT (datetime('now','localtime')),
      FOREIGN KEY (book_id) REFERENCES books(id)
    )
  ''');
  await db.execute('CREATE INDEX IF NOT EXISTS idx_checkin_book_date ON checkin_details(book_id, ch...
  await db.execute('CREATE INDEX IF NOT EXISTS idx_checkin_date ON checkin_details(checkin_date)');
}

/// ■■■■■
Future<void> _onUpgrade(Database db, int oldVersion, int newVersion) async {
  if (oldVersion < 2) {
```

```
    // V1.5 → V2.0
    await db.execute('ALTER TABLE books ADD COLUMN start_date TEXT');
    await db.execute("ALTER TABLE books ADD COLUMN status TEXT DEFAULT 'reading'");
    await db.execute('ALTER TABLE books ADD COLUMN abandoned_at TEXT');
    await db.execute('ALTER TABLE books ADD COLUMN finished_at TEXT');
    await db.execute(
        "UPDATE books SET start_date = read_date, status = 'reading' WHERE status IS NULL");
    await db.execute('CREATE INDEX IF NOT EXISTS idx_status ON books(status)');
    await db.execute('CREATE INDEX IF NOT EXISTS idx_start_date ON books(start_date)');
  }

  if (oldVersion < 3) {
    // V2.0 → V3.0
    await _upgradeV2ToV3(db);
  }

  if (oldVersion < 4) {
    // V3.0 → V3.5: ■■■■■
    await _upgradeV3ToV4(db);
  }

  if (oldVersion < 5) {
    // V3.5: ■■ checkin_details■■■■■■■■■■■■
    await _upgradeV3ToV4(db);
  }
}

/// V3.0 → V3.5 ■■■■■ checkin_details ■
Future<void> _upgradeV3ToV4(Database db) async {
  await db.execute('''
    CREATE TABLE IF NOT EXISTS checkin_details (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      book_id INTEGER NOT NULL,
      checkin_date TEXT NOT NULL,
      duration_min INTEGER,
      note TEXT,
      created_at TEXT DEFAULT (datetime('now','localtime')),
      FOREIGN KEY (book_id) REFERENCES books(id)
    )
  ''');

  // ■■
  await db.execute('CREATE INDEX IF NOT EXISTS idx_checkin_book_date ON checkin_details(book_id, ch...
  await db.execute('CREATE INDEX IF NOT EXISTS idx_checkin_date ON checkin_details(checkin_date)');
}

/// V2.0 → V3.0 ■■
Future<void> _upgradeV2ToV3(Database db) async {
  // Step 1: ■■■■
  await db.execute('ALTER TABLE books RENAME TO books_v2');
```

```
// Step 2: ■■■■■■■■ wish/reading/done + read_count■
await db.execute('''
  CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL DEFAULT '',
    cover_path TEXT,
    rating INTEGER,
    notes TEXT,
    read_date DATE,
    start_date TEXT NOT NULL,
    status TEXT DEFAULT 'wish' CHECK(status IN ('wish', 'reading', 'done', 'abandoned')),
    abandoned_at DATETIME,
    finished_at DATETIME,
    read_count INTEGER NOT NULL DEFAULT 1,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  )
''');

// Step 3: ■■■■
// finished → done, ■■ status ■■■■
// author: ■■ null → ''
// read_count: ■■ 1
await db.execute('''
  INSERT INTO books (id, title, author, cover_path, rating, notes, read_date, start_date, status,...
  SELECT
    id,
    title,
    COALESCE(author, '') as author,
    cover_path,
    rating,
    notes,
    read_date,
    start_date,
    CASE WHEN status = 'finished' THEN 'done' ELSE COALESCE(status, 'reading') END as status,
    abandoned_at,
    finished_at,
    1 as read_count,
    COALESCE(created_at, datetime('now')) as created_at
  FROM books_v2
''');

// Step 4: ■■■■
await db.execute('DROP TABLE books_v2');

// Step 5: ■■■■
await db.execute('CREATE INDEX IF NOT EXISTS idx_status ON books(status)');
await db.execute('CREATE INDEX IF NOT EXISTS idx_read_date ON books(read_date)');
await db.execute('CREATE INDEX IF NOT EXISTS idx_start_date ON books(start_date)');
```

```
    await db.execute('CREATE INDEX IF NOT EXISTS idx_finish_date ON books(finished_at)');
    await db.execute('CREATE INDEX IF NOT EXISTS idx_author ON books(author)');

    // Step 6: ■■ year_goals ■
    await db.execute('''
      CREATE TABLE IF NOT EXISTS year_goals (
        year INTEGER PRIMARY KEY,
        target INTEGER NOT NULL DEFAULT 0,
        is_set_by_user INTEGER NOT NULL DEFAULT 0
      )
    ''');

    // Step 7: ■■■■■■
    final year = DateTime.now().year;
    await db.execute('''
      INSERT OR IGNORE INTO year_goals (year, target, is_set_by_user)
      VALUES ($year,
        COALESCE((SELECT CAST(value AS INTEGER) FROM settings WHERE key = 'yearly_goal'), 0),
        0)
    ''');

    // Step 8: ■■■■■■■■■■V3.4: ■■■■■■0■
    await db.execute("UPDATE settings SET value = '0' WHERE key = 'yearly_goal' AND value = '50'");
    await db.execute("UPDATE settings SET value = '0' WHERE key = 'yearly_goal' AND value = '52'");
  }

  /// ■■■■■
  Future<void> close() async {
    final db = _database;
    if (db != null) {
      await db.close();
      _database = null;
    }
  }
}


import 'package:sqflite/sqflite.dart';

/// V1.5 → V2.0 ■■■■■■■
class V2Migration {
  /// ■■■■
  static Future<void> migrate(Database db) async {
    await db.execute('ALTER TABLE books ADD COLUMN start_date TEXT');
    await db.execute(
        "ALTER TABLE books ADD COLUMN status TEXT DEFAULT 'finished'");
    await db.execute('ALTER TABLE books ADD COLUMN abandoned_at TEXT');
    await db.execute('ALTER TABLE books ADD COLUMN finished_at TEXT');

    // ■■■■■start_date = read_date■status = finished
```

```
    await db.execute(
        "UPDATE books SET start_date = read_date, status = 'finished' WHERE status IS NULL");

    // ■■■■
    await db.execute('CREATE INDEX IF NOT EXISTS idx_status ON books(status)');
    await db.execute(
        'CREATE INDEX IF NOT EXISTS idx_start_date ON books(start_date)');
  }

  /// ■■■■■■■■■■
  static Future<void> rollback(Database db) async {
    await db.execute('''
      CREATE TABLE books_v1 AS
      SELECT id, title, author, read_date, rating, notes, cover_path, created_at
      FROM books
    ''');
    await db.execute('DROP TABLE books');
    await db.execute('ALTER TABLE books_v1 RENAME TO books');
  }
}


import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:provider/provider.dart';
import 'databases/database_helper.dart';
import 'repositories/book_repository.dart';
import 'repositories/settings_repository.dart';
import 'repositories/goal_repository.dart';
import 'repositories/checkin_repository.dart';
import 'providers/books_provider.dart';
import 'providers/filter_provider.dart';
import 'providers/settings_provider.dart';
import 'providers/purchase_provider.dart';
import 'providers/theme_provider.dart';
import 'providers/checkin_provider.dart';
import 'services/notification_service.dart';
import 'services/reminder_scheduler.dart';
import 'theme/app_theme.dart';
import 'routes/route_generator.dart';
import 'routes/app_routes.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();

  // ■■■■■■■■■■■■■■■■■■■■■■ async ■■
  final dbHelper = DatabaseHelper.instance;
  final bookRepository = BookRepository(dbHelper);
  final settingsRepository = SettingsRepository(dbHelper);
  final goalRepository = GoalRepository(dbHelper);
```

```
  // [■■] ■■■■■■■■■■■■■■■■■■■■
  // ■■ Flutter ■■■■■■■■■ SplashPage■■■■■■■■

  runApp(
    _DelayedInitApp(
      bookRepository: bookRepository,
      settingsRepository: settingsRepository,
      goalRepository: goalRepository,
      checkinRepository: CheckinRepository(dbHelper),
    ),
  );
}

/// ■■■■■■■■■ SplashPage■■■■■■■■■■■■■■
class _DelayedInitApp extends StatelessWidget {
  final BookRepository bookRepository;
  final SettingsRepository settingsRepository;
  final GoalRepository goalRepository;
  final CheckinRepository checkinRepository;

  const _DelayedInitApp({
    required this.bookRepository,
    required this.settingsRepository,
    required this.goalRepository,
    required this.checkinRepository,
  });

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (_) => CheckinProvider(
            repository: checkinRepository,
            bookRepository: bookRepository,
          ),
        ),
        ChangeNotifierProvider(
          create: (_) => BooksProvider(
            repository: bookRepository,
            settingsRepository: settingsRepository,
          ),
        ),
        ChangeNotifierProvider(
          create: (_) => SettingsProvider(
            repository: settingsRepository,
          ),
        ),
        ChangeNotifierProvider(
```

```
          create: (_) => PurchaseProvider(
            repository: settingsRepository,
          ),
        ),
        ChangeNotifierProvider(
          create: (_) => FilterProvider(
            bookRepository: bookRepository,
            settingsRepository: settingsRepository,
          ),
        ),
        ChangeNotifierProvider(
          create: (_) => ThemeProvider(
            repository: settingsRepository,
          ),
        ),
      ],
      child: _DelayedInitWrapper(
        bookRepository: bookRepository,
        settingsRepository: settingsRepository,
      ),
    );
  }
}

/// ■■■■■■■■■■■■
class _DelayedInitWrapper extends StatefulWidget {
  final BookRepository bookRepository;
  final SettingsRepository settingsRepository;

  const _DelayedInitWrapper({
    required this.bookRepository,
    required this.settingsRepository,
  });

  @override
  State<_DelayedInitWrapper> createState() => _DelayedInitWrapperState();
}

class _DelayedInitWrapperState extends State<_DelayedInitWrapper> {
  @override
  void initState() {
    super.initState();

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    WidgetsBinding.instance.addPostFrameCallback((_) {
      Future.microtask(_initAfterFirstFrame);
    });
  }

  Future<void> _initAfterFirstFrame() async {
```

```
    // ■■■■■■■■
    await NotificationService().init();

    if (!mounted) return;

    // ■■■■
    context.read<CheckinProvider>().loadMonthCheckins();
    context.read<BooksProvider>().loadBooks();
    context.read<FilterProvider>().loadInitial();
    context.read<SettingsProvider>().loadSettings();
    context.read<PurchaseProvider>().loadPurchaseStatus();
    context.read<ThemeProvider>().loadTheme();

    // ■■■■■■■■■
    Future.microtask(() async {
      if (!mounted) return;
      final booksProvider = context.read<BooksProvider>();
      final settingsProvider = context.read<SettingsProvider>();
      await ReminderScheduler().restoreAfterStartup(
        booksProvider: booksProvider,
        settingsProvider: settingsProvider,
      );
    });
  }

  @override
  Widget build(BuildContext context) {
    return Consumer<ThemeProvider>(
      builder: (context, themeProvider, _) {
        return MaterialApp(
          title: '■■■■■',
          debugShowCheckedModeBanner: false,
          theme: AppTheme.lightTheme,
          darkTheme: AppTheme.darkTheme,
          themeMode: themeProvider.isDark ? ThemeMode.dark : ThemeMode.light,
          localizationsDelegates: const [
            GlobalMaterialLocalizations.delegate,
            GlobalWidgetsLocalizations.delegate,
            GlobalCupertinoLocalizations.delegate,
          ],
          supportedLocales: const [
            Locale('zh', 'CN'),
            Locale('en', 'US'),
          ],
          locale: const Locale('zh', 'CN'),
          initialRoute: AppRoutes.splash,
          onGenerateRoute: RouteGenerator.generateRoute,
        );
      },
    );
```

```
  }
}


/// ■■■■■■■■■■V3.0■
enum BookStatus {
  /// ■■■■ V3.0■
  wish('wish'),

  /// ■■
  reading('reading'),

  /// ■■■■V2.0 finished ■■■ done■
  done('done'),

  /// ■■■■■■■■■■
  abandoned('abandoned');

  final String value;
  const BookStatus(this.value);

  static BookStatus fromString(String value) {
    return BookStatus.values.firstWhere(
      (e) => e.value == value,
      orElse: () => BookStatus.wish,
    );
  }
}

/// ■■■■■■■V3.0■
class Book {
  final int? id;
  final String title;
  final String author; // V3.0: ■■
  final String? coverPath;
  final double? rating; // V3.0: double ■■■0.5 ■■
  final String? notes; // V3.0: max 500 ■■■■■■■
  final DateTime? readDate;
  final DateTime startDate;
  final BookStatus status; // V3.0: wish / reading / done
  final DateTime? abandonedAt;
  final DateTime? finishedAt;
  final int readCount; // ■ V3.0: ■■■■
  final DateTime? createdAt;

  const Book({
    this.id,
    required this.title,
    this.author = '',
    this.coverPath,
```

```
  this.rating,
  this.notes,
  this.readDate,
  required this.startDate,
  this.status = BookStatus.wish,
  this.abandonedAt,
  this.finishedAt,
  this.readCount = 1,
  this.createdAt,
});

// ============ ■■■■ ============

/// ■■■■■■■■■■■■■
int get elapsedDays => DateTime.now().difference(startDate).inDays;

/// ■■■■■■■■■■■■■
int? get readingCycleDays =>
    readDate != null ? readDate!.difference(startDate).inDays : null;

/// ■■■■■■■■■2026■5■8■
String get formattedStartDate =>
    '${startDate.year}■${startDate.month}■${startDate.day}■';

/// ■■■■■■■■■2026■5■8■
String? get formattedReadDate => readDate != null
    ? '${readDate!.year}■${readDate!.month}■${readDate!.day}■'
    : null;

// ============ ■■■ ============

/// ■■■■ Map ■■■V3.0: rating ■ int(0-50) ■■ double(0.0-5.0)■
factory Book.fromMap(Map<String, dynamic> map) {
  final rawRating = map['rating'] as int?;
  return Book(
    id: map['id'] as int?,
    title: map['title'] as String,
    author: map['author'] as String? ?? '',
    coverPath: map['cover_path'] as String?,
    rating: rawRating != null ? rawRating / 10.0 : null,
    notes: map['notes'] as String?,
    readDate: map['read_date'] != null
        ? DateTime.tryParse(map['read_date'] as String)
        : null,
    startDate: DateTime.parse(map['start_date'] as String),
    status: BookStatus.fromString(map['status'] as String? ?? 'reading'),
    abandonedAt: map['abandoned_at'] != null
        ? DateTime.tryParse(map['abandoned_at'] as String)
        : null,
    finishedAt: map['finished_at'] != null
```

```
        ? DateTime.tryParse(map['finished_at'] as String)
        : null,
    readCount: map['read_count'] as int? ?? 1,
    createdAt: map['created_at'] != null
        ? DateTime.tryParse(map['created_at'] as String)
        : null,
  );
}

/// ■■■■■■ Map■V3.0: rating ■ double ■ int*10 ■■■
Map<String, dynamic> toMap() {
  return {
    if (id != null) 'id': id,
    'title': title,
    'author': author,
    if (coverPath != null) 'cover_path': coverPath,
    if (rating != null) 'rating': (rating! * 10).round(),
    if (notes != null) 'notes': notes,
    if (readDate != null) 'read_date': _formatDate(readDate!),
    'start_date': _formatDate(startDate),
    'status': status.value,
    if (abandonedAt != null) 'abandoned_at': abandonedAt!.toIso8601String(),
    if (finishedAt != null) 'finished_at': finishedAt!.toIso8601String(),
    'read_count': readCount,
  };
}

/// ■■■■
Book copyWith({
  int? id,
  String? title,
  String? author,
  String? coverPath,
  double? rating,
  String? notes,
  DateTime? readDate,
  DateTime? startDate,
  BookStatus? status,
  DateTime? abandonedAt,
  DateTime? finishedAt,
  int? readCount,
  DateTime? createdAt,
  bool clearId = false,
  bool clearReadDate = false,
  bool clearRating = false,
  bool clearNotes = false,
  bool clearCoverPath = false,
  bool clearAuthor = false,
}) {
  return Book(
```

```
      id: clearId ? null : (id ?? this.id),
      title: title ?? this.title,
      author: clearAuthor ? '' : (author ?? this.author),
      coverPath: clearCoverPath ? null : (coverPath ?? this.coverPath),
      rating: clearRating ? null : (rating ?? this.rating),
      notes: clearNotes ? null : (notes ?? this.notes),
      readDate: clearReadDate ? null : (readDate ?? this.readDate),
      startDate: startDate ?? this.startDate,
      status: status ?? this.status,
      abandonedAt: abandonedAt ?? this.abandonedAt,
      finishedAt: finishedAt ?? this.finishedAt,
      readCount: readCount ?? this.readCount,
      createdAt: createdAt ?? this.createdAt,
    );
  }

  @override
  String toString() =>
      'Book(id: $id, title: $title, status: ${status.value}, author: $author, readCount: $readCount)';

  /// ■■■■■■■ YYYY-MM-DD
  static String _formatDate(DateTime date) =>
      '${date.year}-${date.month.toString().padLeft(2, '0')}-${date.day.toString().padLeft(2, '0')}';
}


/// ■■■■■■■■■■V3.5■
/// ■■ checkin_details ■■■■■■■■■■■
class CheckinDetail {
  final int? id;
  final int bookId;
  final String checkinDate; // 'YYYY-MM-DD'
  final int? durationMin; // ■■■■■■■■■■■■
  final String? note; // ■■■■■■■■200■■
  final DateTime? createdAt;

  const CheckinDetail({
    this.id,
    required this.bookId,
    required this.checkinDate,
    this.durationMin,
    this.note,
    this.createdAt,
  });

  // ============ ■■■■ ============

  /// ■■■■■■■■ "1■■30■■"
  String get formattedDuration {
    if (durationMin == null) return '';
```

```
  final h = durationMin! ~/ 60;
  final m = durationMin! % 60;
  if (h == 0 && m == 0) return '';
  if (h == 0) return '${m}■■';
  if (m == 0) return '${h}■■';
  return '${h}■■${m}■■';
}

/// ■■■■■■■■ "2026■6■28■"
String get formattedDate {
  final parts = checkinDate.split('-');
  if (parts.length != 3) return checkinDate;
  return '${int.parse(parts[0])}■${int.parse(parts[1])}■${int.parse(parts[2])}■';
}

/// ■■■■■
String get weekday {
  final parts = checkinDate.split('-');
  final date = DateTime(int.parse(parts[0]), int.parse(parts[1]), int.parse(parts[2]));
  const weekdays = ['■■■', '■■■', '■■■', '■■■', '■■■', '■■■', '■■■'];
  return weekdays[date.weekday - 1];
}

// ============ ■■■ ============

factory CheckinDetail.fromMap(Map<String, dynamic> map) {
  return CheckinDetail(
    id: map['id'] as int?,
    bookId: map['book_id'] as int,
    checkinDate: map['checkin_date'] as String,
    durationMin: map['duration_min'] as int?,
    note: map['note'] as String?,
    createdAt: map['created_at'] != null
        ? DateTime.tryParse(map['created_at'] as String)
        : null,
  );
}

Map<String, dynamic> toMap() {
  return {
    if (id != null) 'id': id,
    'book_id': bookId,
    'checkin_date': checkinDate,
    if (durationMin != null) 'duration_min': durationMin,
    if (note != null && note!.isNotEmpty) 'note': note,
  };
}

CheckinDetail copyWith({
  int? id,
```

```
    int? bookId,
    String? checkinDate,
    int? durationMin,
    String? note,
    DateTime? createdAt,
    bool clearDuration = false,
    bool clearNote = false,
  }) {
    return CheckinDetail(
      id: id ?? this.id,
      bookId: bookId ?? this.bookId,
      checkinDate: checkinDate ?? this.checkinDate,
      durationMin: clearDuration ? null : (durationMin ?? this.durationMin),
      note: clearNote ? null : (note ?? this.note),
      createdAt: createdAt ?? this.createdAt,
    );
  }

  @override
  String toString() =>
      'CheckinDetail(id: $id, bookId: $bookId, date: $checkinDate, durationMin: $durationMin)';
}

/// ■■■■■■■■■■■■■ Dialog ■■■■■■
class CheckinDaySummary {
  /// ■■■■■■■■■
  final List<CheckinDetail> details;

  /// ■■■■■■■■■■bookId → title■
  final Map<int, String> bookTitles;

  CheckinDaySummary({
    required this.details,
    required this.bookTitles,
    this.streakDays = 0,
  });

  /// ■■■■■■■■■■
  int get totalMinutes =>
      details.fold(0, (sum, d) => sum + (d.durationMin ?? 0));

  /// ■■■■■■■
  String get formattedTotalDuration {
    final h = totalMinutes ~/ 60;
    final m = totalMinutes % 60;
    if (h == 0 && m == 0) return '';
    if (h == 0) return '${m}■■';
    if (m == 0) return '${h}■■';
    return '${h}■■${m}■■';
  }
```

```
  /// ■■■■■■■■■■■■■
  final int streakDays;

  /// ■■■■■■■■
  bool get hasNote =>
      details.any((d) => d.note != null && d.note!.isNotEmpty);
}


/// ■■■■■■V3.0 ■■ — ■■■■ & ■■■■■■■
class FilterState {
  /// null = ■■■■■■Pro■■■■■
  int? selectedYear;

  /// null = ■■■■■■■■■■■■■■■■■■■■■
  int? selectedMonth;

  /// ■■ feature flag
  bool isPro;

  /// V3.1 ■■■■■■■■■■■■■■■■■■■■■■■■■
  bool fromReadingLife;

  FilterState({
    this.selectedYear,
    this.selectedMonth,
    this.isPro = false,
    this.fromReadingLife = false,
  });

  /// ■■■■■■■■■■■■■■+■■■■■■■■■■■
  bool get showProgressRing =>
      selectedYear == DateTime.now().year && selectedMonth == null;

  /// ■■■■■■■■■■■Pro ■■■■■
  bool get isAllYears => selectedYear == null;

  /// ■■■■■■■■
  bool get hasMonth => selectedMonth != null;

  /// ■■■■■
  String get dynamicTitle {
    final year = selectedYear;
    final month = selectedMonth;

    if (year == null && month == null) return '■ ■■■■■■■';
    if (year == DateTime.now().year && month == null) return '■ $year ■■■■■■';
    if (year != null && month == null) return '■ $year ■■■■■■';
    if (year == null && month != null) return '■ ■■■■■■■ · ${month}■';
```

```
    return '■ $year · $month■';
  }

  /// ■■
  FilterState copyWith({
    int? Function()? selectedYear,
    int? Function()? selectedMonth,
    bool? isPro,
    bool? fromReadingLife,
  }) {
    return FilterState(
      selectedYear: selectedYear != null ? selectedYear() : this.selectedYear,
      selectedMonth: selectedMonth != null ? selectedMonth() : this.selectedMonth,
      isPro: isPro ?? this.isPro,
      fromReadingLife: fromReadingLife ?? this.fromReadingLife,
    );
  }
}


/// ■■■■■■
class ReadingStats {
  /// ■■■■■■■■
  final int currentYearFinished;

  /// ■■■■■■
  final int currentMonthReading;

  /// ■■■■■■■ -> ■■■
  final Map<int, int> monthlyTrend;

  /// ■■■■■■■■■
  final double? avgCycleDays;

  /// ■■■■■■■■■
  final int? fastestCycleDays;

  /// ■■■■■■■■■
  final int? slowestCycleDays;

  /// ■■■■ TOP5■■■■ -> ■■■
  final List<AuthorStat> topAuthors;

  /// ■■■■
  final double? avgRating;

  const ReadingStats({
    this.currentYearFinished = 0,
    this.currentMonthReading = 0,
    this.monthlyTrend = const {},
```

```
    this.avgCycleDays,
    this.fastestCycleDays,
    this.slowestCycleDays,
    this.topAuthors = const [],
    this.avgRating,
  });
}

/// ■■■■
class AuthorStat {
  final String author;
  final int count;

  const AuthorStat({required this.author, required this.count});
}

/// ■■■■■■
class YearlyComparison {
  final int year;
  final int count;
  final double? avgCycle;

  const YearlyComparison({
    required this.year,
    required this.count,
    this.avgCycle,
  });
}

/// ■■■■■■■■
class CareerStats {
  final int totalBooks;
  final int totalDays;
  final int authorCount;
  final double? avgRating;
  final DateTime? firstReadDate;
  final int? longestStreak;
  final DateTime? streakStart;
  final DateTime? streakEnd;
  final double? avgCycleDays;
  final List<YearlyComparison> yearlyComparison;
  final List<AuthorStat> topAuthors;

  const CareerStats({
    this.totalBooks = 0,
    this.totalDays = 0,
    this.authorCount = 0,
    this.avgRating,
    this.firstReadDate,
    this.longestStreak,
```

```
    this.streakStart,
    this.streakEnd,
    this.avgCycleDays,
    this.yearlyComparison = const [],
    this.topAuthors = const [],
  });
}


/// ■■■■■■■
enum SettingKey {
  yearlyGoal('yearly_goal', '0'),
  theme('theme', 'light'),
  dailyReminder('daily_reminder', 'false'),
  reminderTime('reminder_time', '21:00'),
  proPurchased('pro_purchased', 'false'),
  backupEnabled('backup_enabled', 'false');

  final String key;
  final String defaultValue;

  const SettingKey(this.key, this.defaultValue);
}


/// ■■■■■■■■■V3.0 ■■■
class YearGoal {
  final int year;
  int target;
  bool isSetByUser;

  YearGoal({
    required this.year,
    this.target = 0,
    this.isSetByUser = false,
  });

  factory YearGoal.fromMap(Map<String, dynamic> map) {
    return YearGoal(
      year: map['year'] as int,
      target: map['target'] as int? ?? 0,
      isSetByUser: (map['is_set_by_user'] as int? ?? 0) == 1,
    );
  }

  Map<String, dynamic> toMap() => {
        'year': year,
        'target': target,
        'is_set_by_user': isSetByUser ? 1 : 0,
      };
```

```
  @override
  String toString() => 'YearGoal(year: $year, target: $target, setByUser: $isSetByUser)';
}


import 'dart:io';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:image_picker/image_picker.dart';
import '../models/book.dart';
import '../providers/books_provider.dart';
import '../providers/purchase_provider.dart';
import '../theme/colors.dart';
import '../constants/app_constants.dart';
import '../utils/date_utils.dart' as date_utils;
import '../routes/app_routes.dart';

/// ■■■■■■ / ■■■■■V3.2■
/// - ■■■■■■■■ initialTitle / initialAuthor■■■■
/// - ■■■■■■■■ editBook■■■■ Book ■■■■■■■■■■■■
class AddBookPage extends StatefulWidget {
  final String? initialTitle;
  final String? initialAuthor;
  final Book? editBook; // V3.2 ■■■■
  final bool isFromWish; // V3.5■■■■■■■■■■■■■■■■■■■■■■■■

  const AddBookPage({super.key, this.initialTitle, this.initialAuthor, this.editBook, this.isFromWish...

  @override
  State<AddBookPage> createState() => _AddBookPageState();
}

class _AddBookPageState extends State<AddBookPage> {
  final _formKey = GlobalKey<FormState>();

  // ■■■■■■■■ book■■■■■■■■■■■■
  Book? get _editBook => widget.editBook;

  late final TextEditingController _titleController;
  late final TextEditingController _authorController;
  final _titleFocusNode = FocusNode();
  final _authorFocusNode = FocusNode();
  late DateTime _startDate;
  String? _coverPath;
  bool _isSaving = false;
  bool _isDeleting = false;

  /// ■■■■■■■■■■
  bool get _isEditing => _editBook != null;
```

```
@override
void initState() {
  super.initState();
  // ■■■■■■■■■■■■■■■■■■■■
  final edit = widget.editBook;
  _titleController = TextEditingController(text: widget.initialTitle ?? edit?.title ?? '');
  _authorController = TextEditingController(text: widget.initialAuthor ?? edit?.author ?? '');
  _startDate = edit?.startDate ?? DateTime.now();
  _coverPath = edit?.coverPath;
}

@override
void dispose() {
  _titleFocusNode.dispose();
  _authorFocusNode.dispose();
  _titleController.dispose();
  _authorController.dispose();
  super.dispose();
}

Future<void> _pickCover() async {
  final result = await showModalBottomSheet<ImageSource>(
    context: context,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(16)),
    ),
    builder: (ctx) => Padding(
      padding: const EdgeInsets.all(24),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          const Text(
            '■■■■■■',
            style: TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.w600,
              color: AppColors.textPrimary,
            ),
          ),
          const SizedBox(height: 20),
          ListTile(
            leading: Container(
              width: 40,
              height: 40,
              decoration: BoxDecoration(
                color: const Color(0xFFFFE3E3),
                borderRadius: BorderRadius.circular(8),
              ),
              child: const Icon(Icons.camera_alt, color: AppColors.primary),
```

```
              ),
              title: const Text('■■'),
              onTap: () => Navigator.pop(ctx, ImageSource.camera),
            ),
            ListTile(
              leading: Container(
                width: 40,
                height: 40,
                decoration: BoxDecoration(
                  color: const Color(0xFFFFE3E3),
                  borderRadius: BorderRadius.circular(8),
                ),
                child: const Icon(Icons.photo_library, color: AppColors.primary),
              ),
              title: const Text('■■■■■'),
              onTap: () => Navigator.pop(ctx, ImageSource.gallery),
            ),
          ],
        ),
      ),
    );

    if (result != null) {
      final picker = ImagePicker();
      final picked = await picker.pickImage(source: result);
      if (picked != null) {
        setState(() => _coverPath = picked.path);
      }
    }
  }

  Future<void> _selectDate() async {
    final now = DateTime.now();
    final picked = await showDatePicker(
      context: context,
      initialDate: _startDate,
      firstDate: DateTime(2000),
      lastDate: now.add(const Duration(days: 365)),
      locale: const Locale('zh'),
    );
    if (picked != null) {
      setState(() => _startDate = picked);
    }
  }

  /// ■■■■■■■■addBook■■■■■■updateBook■
  Future<void> _save() async {
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isSaving = true);
```

```
final provider = context.read<BooksProvider>();

if (_isEditing) {
  // ■■■■■■■■ updateBook ■■ id
  final success = await provider.updateBook(
    bookId: _editBook!.id!,
    title: _titleController.text.trim(),
    author: _authorController.text.trim().isEmpty
        ? null
        : _authorController.text.trim(),
    coverPath: _coverPath,
    startDate: _startDate,
    status: _editBook!.status,
  );

  if (mounted) {
    setState(() => _isSaving = false);
    if (success) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('■${_titleController.text.trim()}■■■■')),
      );
      Navigator.pop(context, true);
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('■■■■■■■■■')),
      );
    }
  }
} else if (widget.isFromWish) {
  // ■■■→■■■■■■■■■■■■■■■■■■
  final success = await provider.addBook(
    title: _titleController.text.trim(),
    author: _authorController.text.trim().isEmpty
        ? null
        : _authorController.text.trim(),
    coverPath: _coverPath,
    startDate: _startDate,
  );

  if (mounted) {
    setState(() => _isSaving = false);
    if (success) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('■${_titleController.text.trim()}■■■■■■')),
      );
      Navigator.pop(context, true);
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('■■■■■■■■■')),
```

```
        );
      }
    }
    return;
  } else {
    // ■■■■■■■■■■■■
    final isPro = context.read<PurchaseProvider>().isPro;
    final canAdd = await provider.canAddBook(isPro: isPro);
    if (!canAdd) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('■■■■■■■■ 5 ■■■■■■ Pro ■■■■■■')),
        );
        Navigator.pushNamed(context, AppRoutes.pro);
      }
      setState(() => _isSaving = false);
      return;
    }

    final success = await provider.addBook(
      title: _titleController.text.trim(),
      author: _authorController.text.trim().isEmpty
          ? null
          : _authorController.text.trim(),
      coverPath: _coverPath,
      startDate: _startDate,
    );

    if (mounted) {
      setState(() => _isSaving = false);
      if (success) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('■${_titleController.text.trim()}■■■■■■■')),
        );
        Navigator.pop(context, true);
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('■■■■■■■■■')),
        );
      }
    }
  }
}

/// ■■■■■■■■■■■■■
Future<void> _deleteBook() async {
  if (!_isEditing || _editBook?.id == null) return;

  // ■■■■■■■
  final confirmed = await showDialog<bool>(
```

```
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('■■■■'),
      content: const Text('■■■■■■■■■■■■■■■■■■■■■■'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          style: TextButton.styleFrom(
            backgroundColor: Colors.white,
            padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(8),
              side: const BorderSide(color: Color(0xFFDEE2E6)),
            ),
          ),
          child: const Text('■■',
              style: TextStyle(
                  color: AppColors.textSecondary,
                  fontWeight: FontWeight.w600)),
        ),
        TextButton(
          onPressed: () => Navigator.pop(ctx, true),
          style: TextButton.styleFrom(
            backgroundColor: const Color(0xFFFF6B6B),
            padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(8),
            ),
          ),
          child: const Text('■■■■',
              style: TextStyle(
                  color: Colors.white,
                  fontWeight: FontWeight.w600)),
        ),
      ],
    ),
  );

  if (confirmed == true && mounted) {
    setState(() => _isDeleting = true);
    final provider = context.read<BooksProvider>();
    final success = await provider.deleteBook(_editBook!.id!);
    if (mounted) {
      setState(() => _isDeleting = false);
      if (success) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('■${_editBook!.title}■■■■')),
        );
        Navigator.pop(context, true);
      } else {
```

```
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('■■■■■■■■■')),
        );
      }
    }
  }
}

/// ■■■■■■■■■■■■■■■
String get _pageTitle {
  if (!_isEditing) return '■■■■■';
  switch (_editBook!.status) {
    case BookStatus.wish:
      return '■■■■■';
    case BookStatus.reading:
      return '■■■■■';
    default:
      return '■■■';
  }
}

/// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
String get _primaryButtonLabel {
  if (!_isEditing) return '■■■■■';
  return _editBook!.status == BookStatus.wish ? '■ ■■■■■' : '■ ■■■■■';
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.white,
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      shape: const Border(
        bottom: BorderSide(color: Color(0xFFDEE2E6), width: 1),
      ),
      automaticallyImplyLeading: false,
      title: Row(
        crossAxisAlignment: CrossAxisAlignment.center,
        children: [
          GestureDetector(
            onTap: () => Navigator.pop(context),
            child: const Padding(
              padding: EdgeInsets.only(bottom: 3),
              child: Text(
                '←',
                style: TextStyle(
                  fontSize: 20,
                  color: AppColors.textPrimary,
```

```
              ),
            ),
          ),
        ),
        const SizedBox(width: 8),
        Text(
          _pageTitle,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.w600,
            color: AppColors.textPrimary,
          ),
        ),
      ],
    ),
  ),
  body: Form(
    key: _formKey,
    child: SingleChildScrollView(
      padding: const EdgeInsets.all(20),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // === ■■■■■■===
          _FormLabel(
            text: '■■',
            suffix: '■■■■',
            isOptional: true,
          ),
          const SizedBox(height: 8),
          GestureDetector(
            onTap: _pickCover,
            child: Container(
              width: 120,
              height: 160,
              decoration: BoxDecoration(
                color: const Color(0xFFF8F9FA),
                borderRadius: BorderRadius.circular(12),
                border: Border.all(color: const Color(0xFFDEE2E6), width: 2, strokeAlign: BorderS...
              ),
              child: _coverPath != null
                  ? ClipRRect(
                      borderRadius: BorderRadius.circular(12),
                      child: Image.file(
                        File(_coverPath!),
                        fit: BoxFit.cover,
                      ),
                    )
                  : const Column(
                      mainAxisAlignment: MainAxisAlignment.center,
```

```
              children: [
                Icon(Icons.camera_alt, size: 28, color: Color(0xFFADB5BD)),
                SizedBox(height: 8),
                Text(
                  '■■■■■■■',
                  style: TextStyle(
                    fontSize: 12,
                    color: Color(0xFFADB5BD),
                  ),
                ),
              ],
            ),
  ),
),
const SizedBox(height: 24),

// === ■■■■■■■===
_FormLabel(text: '■■', required: true),
const SizedBox(height: 8),
Stack(
  children: [
    TextField(
      controller: _titleController,
      focusNode: _titleFocusNode,
      textInputAction: TextInputAction.next,
      onTapOutside: (_) => _titleFocusNode.unfocus(),
      decoration: InputDecoration(
        hintText: '■■■■',
        hintStyle: const TextStyle(color: AppColors.textMuted),
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: const BorderSide(color: Color(0xFFDEE2E6)),
        ),
        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: const BorderSide(color: Color(0xFFDEE2E6)),
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: const BorderSide(color: AppColors.primary, width: 2),
        ),
        contentPadding: const EdgeInsets.fromLTRB(16, 14, 16, 28),
        counterText: '',
      ),
      maxLength: AppConstants.maxTitleLength,
      autofocus: true,
    ),
    Positioned(
      right: 12,
      bottom: 8,
```

```
@override
Widget build(BuildContext context) {
  final screenSize = MediaQuery.of(context).size;

  debugPrint('[CelebrationWidget] build: showRing=$_showRingAnimation, showConfetti=$_showConfetti,...

  return Material(
    color: Colors.transparent,
    child: Stack(
      children: [
        // ■■ + ■■■■
        Positioned.fill(
          child: GestureDetector(
            onTap: _dismiss,
            child: Container(color: Colors.black.withValues(alpha: 0.12)),
          ),
        ),
        // ■■■■■■■■■■
      // ■■■■■■■■■■■■■■■ _showConfetti■
      // ■■■■■■■■■■■■■■■■ CustomPaint ■■
      if (_confettiPieces.isNotEmpty)
        Positioned.fill(
          child: IgnorePointer(
            child: Stack(
              children: [
                // ■■■■■■■■■■■■■■■ overlay ■■■■
                Container(color: Colors.blue.withValues(alpha: 0.05)),
                // ■■■■
                CustomPaint(
                  painter: ConfettiPainter(
                    pieces: _confettiPieces,
                    animationValue: _confettiValue,
                    height: screenSize.height,
                  ),
                ),
              ],
            ),
          ),
        ),
        // ■■■■
        if (_showBanner)
          Positioned(
            left: 24,
            right: 24,
            bottom: screenSize.height / 2 - 140,
            child: IgnorePointer(
              ignoring: false,
              child: _buildBanner(),
            ),
```

```
          ),
        ],
      ),
    );
  }

  Widget _buildBanner() {
    debugPrint('[CelebrationWidget] Building banner, showBanner=$_showBanner, confettiValue=$_confett...
    return AnimatedOpacity(
      opacity: _showBanner ? 1.0 : 0.0,
      duration: const Duration(milliseconds: 600),
      curve: const Cubic(0.34, 1.56, 0.64, 1),
      child: Container(
        decoration: BoxDecoration(
          gradient: const LinearGradient(
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
            colors: [Color(0xFFFFD43B), Color(0xFFFF922B)],
          ),
          borderRadius: BorderRadius.circular(20),
          boxShadow: [
            BoxShadow(
              color: const Color(0xFFFF922B).withValues(alpha: 0.35),
              blurRadius: 32,
              offset: const Offset(0, 8),
            ),
          ],
        ),
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('■', style: TextStyle(fontSize: 52)),
            const SizedBox(height: 8),
            const Text(
              '■■■■■■■■■',
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.w700,
                color: Colors.white,
              ),
            ),
            const SizedBox(height: 4),
            const Text(
              '■■■■■■■■■■■■',
              style: TextStyle(
                fontSize: 14,
                color: Color(0xD9FFFFFF),
              ),
            ),
```

```
            const SizedBox(height: 14),
            GestureDetector(
              onTap: () {
                _dismissed = true;
                _ringController.stop();
                _confettiController.stop();
                widget.onShare();
              },
              child: Container(
                padding: const EdgeInsets.symmetric(horizontal: 28, vertical: 10),
                decoration: BoxDecoration(
                  color: Colors.white.withValues(alpha: 0.2),
                  border: Border.all(
                    color: Colors.white.withValues(alpha: 0.5),
                    width: 1.5,
                  ),
                  borderRadius: BorderRadius.circular(24),
                ),
                child: const Text(
                  '■ ■■■■',
                  style: TextStyle(
                    fontSize: 15,
                    fontWeight: FontWeight.w600,
                    color: Colors.white,
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}


import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/checkin_provider.dart';
import '../providers/books_provider.dart';
import '../models/checkin.dart';
import '../theme/colors.dart';
import 'checkin_dialog.dart';
import 'checkin_option_sheet.dart';

/// ■■■■■■■V3.5■
/// ■■■ + ■■■■ + ■■■■■■ Dialog
class CheckinCalendar extends StatefulWidget {
  const CheckinCalendar({super.key});
```

```
  @override
  State<CheckinCalendar> createState() => _CheckinCalendarState();
}

class _CheckinCalendarState extends State<CheckinCalendar> {
  // ============ ■■■■■■ ============

  /// ■■■■■■■■■■
  /// ■■ List<List<int?>>■■■■■■■■■■null = ■■■■
  List<List<int?>> _buildDateGrid(int year, int month) {
    final firstDay = DateTime(year, month, 1);
    final lastDay = DateTime(year, month + 1, 0);
    final totalDays = lastDay.day;
    // 1=■■ ... 7=■■
    final startWeekday = firstDay.weekday;

    final grid = <List<int?>>[];
    List<int?> week = [];

    // ■■■■■■■1■■■■
    for (int i = 1; i < startWeekday; i++) {
      week.add(null);
    }

    for (int day = 1; day <= totalDays; day++) {
      week.add(day);
      if (week.length == 7) {
        grid.add(week);
        week = [];
      }
    }

    // ■■■■■■
    if (week.isNotEmpty) {
      while (week.length < 7) {
        week.add(null);
      }
      grid.add(week);
    }

    return grid;
  }

  // ============ Dialog ■■ ============

  void _onDateTap(int day) {
    final now = DateTime.now();
    final provider = context.read<CheckinProvider>();
    final year = provider.currentYear;
    final month = provider.currentMonth;
```

```
    final today = DateTime(now.year, now.month, now.day);
    final tappedDate = DateTime(year, month, day);

    // ■■■■■■■■■
    if (tappedDate.isAfter(today)) return;

    final dateStr =
        '${year}-${month.toString().padLeft(2, '0')}-${day.toString().padLeft(2, '0')}';
    final isToday = tappedDate == today;
    final isCheckedIn = provider.checkinDates.contains(dateStr);

    if (isToday) {
      if (isCheckedIn) {
        // ■■■■■■ → ■■■■■
        _showOptionSheet(provider, dateStr);
      } else {
        // ■■■■■■ → ■■■■■ Dialog
        _showNewDialog(provider, dateStr);
      }
    } else if (isCheckedIn) {
      // ■■■■■■ → ■■ Dialog
      _showDetailDialog(provider, dateStr);
    }
    // ■■■■■■ → ■■■
  }

  // ============ Dialog ■■ ============

  void _showNewDialog(CheckinProvider provider, String dateStr, {int? initialBookId}) {
    final booksProvider = context.read<BooksProvider>();
    final readingBooks = booksProvider.readingBooks;
    showModalBottomSheet(
      context: context,
      isScrollControlled: true,
      backgroundColor: Colors.transparent,
      builder: (_) => CheckinNewDialog(
        dateStr: dateStr,
        readingBooks: readingBooks,
        initialBookId: initialBookId,
        onSubmit: (bookId, durationMin, note) async {
          final error = await provider.addCheckin(
            bookId: bookId,
            durationMin: durationMin,
            note: note,
          );
          if (error == null) {
            _showToast('■■■■■ ■■');
          } else {
            _showToast(error);
          }
```

```
      },
    ),
  );
}

void _showOptionSheet(CheckinProvider provider, String dateStr) {
  final lastCheckin = provider.lastCheckin;

  showModalBottomSheet(
    context: context,
    backgroundColor: Colors.transparent,
    builder: (_) => CheckinOptionSheet(
      dateStr: dateStr,
      onNewCheckin: () {
        Navigator.pop(context); // close option sheet
        _showNewDialog(provider, dateStr,
            initialBookId: lastCheckin?.bookId);
      },
      onViewDetail: () {
        Navigator.pop(context); // close option sheet
        _showDetailDialog(provider, dateStr);
      },
    ),
  );
}

void _showDetailDialog(CheckinProvider provider, String dateStr) {
  // ■■■ Future■■■ builder ■■■■■■■■■■■
  final future = provider.getDaySummary(dateStr);
  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    backgroundColor: Colors.transparent,
    builder: (_) => FutureBuilder<CheckinDaySummary>(
      future: future,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        }
        if (!snapshot.hasData) {
          return const Center(child: Text('■■■■'));
        }
        return CheckinDetailDialog(summary: snapshot.data!);
      },
    ),
  );
}

void _showToast(String msg) {
  if (!mounted) return;
```

```
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(msg),
      duration: const Duration(seconds: 2),
      behavior: SnackBarBehavior.floating,
    ),
  );
}

// ============ Build ============

@override
Widget build(BuildContext context) {
  return Consumer<CheckinProvider>(
    builder: (context, provider, _) {
      final year = provider.currentYear;
      final month = provider.currentMonth;
      final grid = _buildDateGrid(year, month);
      final now = DateTime.now();
      final today = DateTime(now.year, now.month, now.day);
      final streakDays = provider.streakDays;

      return Padding(
        padding: const EdgeInsets.symmetric(horizontal: 20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // ■■■■■ + ■■■■ + ■■■■
            Row(
              children: [
                const Text('■', style: TextStyle(fontSize: 16)),
                const SizedBox(width: 4),
                const Text(
                  '■■■■',
                  style: TextStyle(
                    fontSize: 15,
                    fontWeight: FontWeight.w600,
                    color: AppColors.textPrimary,
                  ),
                ),
                if (streakDays > 0) ...[
                  const SizedBox(width: 6),
                  Text(
                    '■ ■■ $streakDays ■',
                    style: const TextStyle(
                      fontSize: 12,
                      fontWeight: FontWeight.w500,
                      color: AppColors.primary,
                    ),
                  ),
```

```
    ],
    const Spacer(),
    Row(
      mainAxisSize: MainAxisSize.min,
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
        GestureDetector(
          onTap: () => provider.goToPrevMonth(),
          child: const Text(
            '‹',
            style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.w700,
              color: AppColors.textPrimary,
            ),
          ),
        ),
        const SizedBox(width: 12),
        Text(
          provider.monthLabel,
          style: const TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.w500,
            color: AppColors.textSecondary,
          ),
        ),
        const SizedBox(width: 12),
        GestureDetector(
          onTap: () => provider.goToNextMonth(),
          child: const Text(
            '›',
            style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.w700,
              color: AppColors.textPrimary,
            ),
          ),
        ),
      ],
    ),
  ],
),
const SizedBox(height: 14),
// ■■■
Row(
  mainAxisAlignment: MainAxisAlignment.spaceAround,
  children: ['■', '■', '■', '■', '■', '■', '■']
      .map(
        (d) => SizedBox(
          width: 34,
```

```
            child: Text(
              d,
              textAlign: TextAlign.center,
              style: const TextStyle(
                fontSize: 11,
                fontWeight: FontWeight.w500,
                color: AppColors.textSecondary,
              ),
            ),
          ),
        )
        .toList(),
),
const SizedBox(height: 4),
// ■■■■
...grid.map((week) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 2),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceAround,
      children: week.map((day) {
        if (day == null) {
          return const SizedBox(
            width: 34,
            height: 34,
          );
        }

        final date = DateTime(year, month, day);
        final dateStr =
            '$year-${month.toString().padLeft(2, '0')}-${day.toString().padLeft(2, '0')}';
        final isToday = date == today;
        final isCheckedIn =
            provider.checkinDates.contains(dateStr);
        final isFuture = date.isAfter(today);

        Color bgColor = Colors.transparent;
        Color textColor = AppColors.textPrimary;
        Color borderColor = Colors.transparent;
        FontWeight fontWeight = FontWeight.w500;
        bool isClickable = true;

        if (isFuture) {
          textColor = AppColors.textMuted;
          isClickable = false;
        } else if (isCheckedIn) {
          bgColor = AppColors.primary;
          textColor = Colors.white;
          fontWeight = FontWeight.w600;
        }
```

```
                    if (isToday) {
                      borderColor = AppColors.primary;
                    }

                    return GestureDetector(
                      onTap:
                          isClickable ? () => _onDateTap(day) : null,
                      child: Container(
                        width: 34,
                        height: 34,
                        decoration: BoxDecoration(
                          color: bgColor,
                          shape: BoxShape.circle,
                          border: Border.all(
                            color: borderColor,
                            width: 2,
                          ),
                        ),
                        child: Center(
                          child: Text(
                            '$day',
                            style: TextStyle(
                              fontSize: 13,
                              fontWeight: fontWeight,
                              color: textColor,
                            ),
                          ),
                        ),
                      ),
                    );
                  }).toList(),
                ),
              );
            }),
          ],
        ),
      );
    },
  );
  }
}


import 'package:flutter/material.dart';
import '../models/book.dart';
import '../models/checkin.dart';
import '../theme/colors.dart';

// ============================================================
```

```
// Dialog 1■■■■■■■■■■·■■■■
// ============================================================

/// ■■■■ Dialog■V3.5 ■■■■■■■■■■■■■■■■
class CheckinNewDialog extends StatefulWidget {
  final String dateStr;
  final List<Book> readingBooks;
  final int? initialBookId; // ■■■■■■■■■■■■■■■■
  final Future<void> Function(int bookId, int? durationMin, String? note)
      onSubmit;

  const CheckinNewDialog({
    super.key,
    required this.dateStr,
    required this.readingBooks,
    this.initialBookId,
    required this.onSubmit,
  });

  @override
  State<CheckinNewDialog> createState() => _CheckinNewDialogState();
}

class _CheckinNewDialogState extends State<CheckinNewDialog> {
  int? _selectedBookId;
  int? _selectedHour;
  int _selectedMinute = 0;
  final _noteController = TextEditingController();
  final _noteFocusNode = FocusNode();
  bool _isSubmitting = false;

  @override
  void initState() {
    super.initState();
    // ■■■■■■■■■■■■■■■■■■
    if (widget.initialBookId != null) {
      _selectedBookId = widget.initialBookId;
    }
  }

  @override
  void dispose() {
    _noteController.dispose();
    _noteFocusNode.dispose();
    super.dispose();
  }

  bool get _canSubmit => _selectedBookId != null && !_isSubmitting;

  Future<void> _submit() async {
```

```
  if (!_canSubmit) return;
  setState(() => _isSubmitting = true);

  final totalMin =
      (_selectedHour ?? 0) * 60 + _selectedMinute;
  await widget.onSubmit(
    _selectedBookId!,
    totalMin > 0 ? totalMin : null,
    _noteController.text.trim().isNotEmpty
        ? _noteController.text.trim()
        : null,
  );
  if (context.mounted) {
    Navigator.pop(context);
  }
}

// ■■■■■■■■
String _formatDate(String dateStr) {
  final parts = dateStr.split('-');
  if (parts.length != 3) return dateStr;
  return '${int.parse(parts[0])}■${int.parse(parts[1])}■${int.parse(parts[2])}■';
}

@override
Widget build(BuildContext context) {
  return _DialogContainer(
    child: Column(
      mainAxisSize: MainAxisSize.min,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // ■■■
        _DialogTitle(
          title: '■■ ■■■■',
          dateSubtitle: _formatDate(widget.dateStr),
          onClose: () => Navigator.pop(context),
        ),

        const SizedBox(height: 16),

        // ■■■■■■
        _BookSelector(
          books: widget.readingBooks,
          selectedBookId: _selectedBookId,
          onChanged: (id) => setState(() => _selectedBookId = id),
        ),

        const SizedBox(height: 16),

        // ■■■■■■■
```

```
_DurationSelector(
  selectedHour: _selectedHour,
  selectedMinute: _selectedMinute,
  onHourChanged: (h) => setState(() => _selectedHour = h),
  onMinuteChanged: (m) => setState(() => _selectedMinute = m),
),

const SizedBox(height: 16),

// ■■■■■
_NoteInput(
  controller: _noteController,
  focusNode: _noteFocusNode,
),

const SizedBox(height: 16),

// ■■■■
SizedBox(
  width: double.infinity,
  height: 42,
  child: ElevatedButton(
    onPressed: _canSubmit ? _submit : null,
    style: ElevatedButton.styleFrom(
      backgroundColor: AppColors.primary,
      foregroundColor: Colors.white,
      disabledBackgroundColor: AppColors.border,
      disabledForegroundColor: Colors.white70,
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(21),
      ),
    ),
    child: _isSubmitting
        ? const SizedBox(
            width: 20,
            height: 20,
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: Colors.white,
            ),
          )
        : const Text(
            '■■ ■■',
            style: TextStyle(
              fontSize: 15,
              fontWeight: FontWeight.w700,
            ),
          ),
  ),
```

```
          ),
        ],
      ),
    );
  }
}

// ============================================================
// Dialog 3■■■■■■■■■■■■·■■■
// ============================================================

class CheckinDetailDialog extends StatelessWidget {
  final CheckinDaySummary summary;

  const CheckinDetailDialog({super.key, required this.summary});

  @override
  Widget build(BuildContext context) {
    return _DialogContainer(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // ■■■
          _DialogTitle(
            title: '■ ${_formatDate(summary.details.first.checkinDate)} · '
                '■■■',
            onClose: () => Navigator.pop(context),
          ),

          if (summary.streakDays > 0) ...[
            const SizedBox(height: 4),
            Padding(
              padding: const EdgeInsets.only(left: 4),
              child: Text(
                '■ ■■■■: ${summary.streakDays} ■(■■■■)',
                style: const TextStyle(
                  fontSize: 12,
                  fontWeight: FontWeight.w500,
                  color: AppColors.primary,
                ),
              ),
            ),
          ],

          const SizedBox(height: 14),

          // ■■■■■■■■■■■■■■ spread ■■■■■■■■
          ..._buildRecordList(),
```

```
// ■■■■
if (summary.totalMinutes > 0) ...[
  const Divider(color: Color(0xFFF1F3F5), thickness: 1),
  const SizedBox(height: 8),
  Row(
    children: [
      const Text(
        '■■■■',
        style: TextStyle(
          fontSize: 13,
          color: AppColors.textMuted,
        ),
      ),
      const Spacer(),
      Text(
        summary.formattedTotalDuration,
        style: const TextStyle(
          fontSize: 14,
          fontWeight: FontWeight.w700,
          color: AppColors.textPrimary,
        ),
      ),
    ],
  ),
  const SizedBox(height: 12),
],

// ■■■■
SizedBox(
  width: double.infinity,
  height: 40,
  child: OutlinedButton(
    onPressed: () => Navigator.pop(context),
    style: OutlinedButton.styleFrom(
      foregroundColor: AppColors.textSecondary,
      backgroundColor: const Color(0xFFF1F3F5),
      side: BorderSide.none,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(20),
      ),
    ),
    child: const Text(
      '■■',
      style: TextStyle(
        fontSize: 14,
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
),
```

```
        ],
      ),
    );
  }

  List<Widget> _buildRecordList() {
    return summary.details.map((detail) {
      final bookTitle =
          summary.bookTitles[detail.bookId] ?? '■■■■■';
      return Padding(
        padding: const EdgeInsets.only(bottom: 8),
        child: _RecordCard(
          bookTitle: bookTitle,
          durationStr: detail.formattedDuration,
          note: detail.note,
        ),
      );
    }).toList();
  }

  String _formatDate(String dateStr) {
    final parts = dateStr.split('-');
    if (parts.length != 3) return dateStr;
    return '${int.parse(parts[0])}■${int.parse(parts[1])}■${int.parse(parts[2])}■';
  }
}

// ============================================================
// ■■■■■
// ============================================================

/// Dialog ■■■■■■■■■■■■■
class _DialogContainer extends StatelessWidget {
  final Widget child;

  const _DialogContainer({required this.child});

  @override
  Widget build(BuildContext context) {
    final bottomInset = MediaQuery.of(context).viewInsets.bottom;
    return Container(
      width: double.infinity,
      constraints: BoxConstraints(
        maxHeight: MediaQuery.of(context).size.height * 0.75,
      ),
      decoration: const BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.only(
          topLeft: Radius.circular(20),
          topRight: Radius.circular(20),
```

```
        ),
      ),
      padding: EdgeInsets.fromLTRB(20, 16, 20, 24 + bottomInset),
      child: SingleChildScrollView(
        physics: const ClampingScrollPhysics(),
        child: child,
      ),
    );
  }
}

/// Dialog ■■■
class _DialogTitle extends StatelessWidget {
  final String title;
  final String? dateSubtitle;
  final VoidCallback? onClose;

  const _DialogTitle({
    required this.title,
    this.dateSubtitle,
    this.onClose,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Row(
          children: [
            Text(
              title,
              style: const TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.w700,
                color: AppColors.textPrimary,
              ),
            ),
            const Spacer(),
            if (onClose != null)
              GestureDetector(
                onTap: onClose,
                child: Container(
                  width: 28,
                  height: 28,
                  decoration: const BoxDecoration(
                    color: Color(0xFFF1F3F5),
                    shape: BoxShape.circle,
                  ),
                  child: const Icon(Icons.close, size: 16, color: Color(0xFF868E96)),
                ),
```

```
              ),
          ],
        ),
        if (dateSubtitle != null && dateSubtitle!.isNotEmpty) ...[
          const SizedBox(height: 4),
          Text(
            dateSubtitle!,
            style: const TextStyle(
              fontSize: 12,
              color: AppColors.textMuted,
            ),
          ),
        ],
      ],
    );
  }
}

/// ■■■■■■■■■
class _BookSelector extends StatelessWidget {
  final List<Book> books;
  final int? selectedBookId;
  final ValueChanged<int?> onChanged;

  const _BookSelector({
    required this.books,
    required this.selectedBookId,
    required this.onChanged,
  });

  @override
  Widget build(BuildContext context) {
    final isEmpty = books.isEmpty;

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Row(
          children: [
            Text('■', style: TextStyle(fontSize: 14)),
            SizedBox(width: 4),
            Text(
              '■■■■',
              style: TextStyle(
                fontSize: 13,
                fontWeight: FontWeight.w500,
                color: AppColors.textSecondary,
              ),
            ),
            Text(
```

```
            ' *',
            style: TextStyle(
              fontSize: 13,
              fontWeight: FontWeight.w500,
              color: AppColors.primary,
            ),
          ),
        ],
      ),
      const SizedBox(height: 6),
      Container(
        height: 40,
        decoration: BoxDecoration(
          border: Border.all(
            color: isEmpty ? AppColors.border.withOpacity(0.5) : AppColors.border,
            width: 1.5,
          ),
          borderRadius: BorderRadius.circular(10),
        ),
        padding: const EdgeInsets.symmetric(horizontal: 12),
        child: DropdownButtonHideUnderline(
          child: DropdownButton<int>(
            value: isEmpty ? null : selectedBookId,
            isExpanded: true,
            hint: Text(
              isEmpty ? '■■■■■■■■■■■■' : '■■■■■■■■',
              style: TextStyle(
                fontSize: 14,
                color: isEmpty
                    ? AppColors.textMuted.withOpacity(0.6)
                    : AppColors.textMuted,
                fontStyle: isEmpty ? FontStyle.italic : FontStyle.normal,
              ),
            ),
            items: books.map((book) {
              return DropdownMenuItem(
                value: book.id,
                child: Text(
                  '■ ${book.title}',
                  style: const TextStyle(fontSize: 14),
                  overflow: TextOverflow.ellipsis,
                ),
              );
            }).toList(),
            onChanged: isEmpty ? null : onChanged,
          ),
        ),
      ),
    ],
  );
```

```
  }
}
/// ■■■■■■■■■■■■ + ■■■
class _DurationSelector extends StatelessWidget {
  final int? selectedHour;
  final int selectedMinute;
  final ValueChanged<int?> onHourChanged;
  final ValueChanged<int> onMinuteChanged;

  const _DurationSelector({
    required this.selectedHour,
    required this.selectedMinute,
    required this.onHourChanged,
    required this.onMinuteChanged,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Row(
          children: [
            Text('■', style: TextStyle(fontSize: 14)),
            SizedBox(width: 4),
            Text(
              '■■■■(■■)',
              style: TextStyle(
                fontSize: 13,
                fontWeight: FontWeight.w500,
                color: AppColors.textSecondary,
              ),
            ),
          ],
        ),
        const SizedBox(height: 6),
        Row(
          children: [
            Expanded(
              child: _buildDropdown(
                value: selectedHour,
                hint: '■■',
                items: [null, ...List.generate(24, (i) => i + 1)],
                itemLabel: (v) => v == null ? '0■■' : '$v■■',
                onChanged: onHourChanged,
              ),
            ),
            const SizedBox(width: 10),
            Expanded(
              child: _buildDropdown(
```

```
                value: selectedMinute,
                hint: '■■',
                items: List.generate(13, (i) => i * 5),
                itemLabel: (v) => v == 0 ? '0■■' : '${v}■■',
                onChanged: (v) => onMinuteChanged(v ?? 0),
              ),
            ),
          ],
        ),
      ],
    );
  }

  Widget _buildDropdown<T>({
    required T? value,
    required String hint,
    required List<T?> items,
    required String Function(T?) itemLabel,
    required ValueChanged<T?> onChanged,
  }) {
    return Container(
      height: 40,
      decoration: BoxDecoration(
        border: Border.all(color: AppColors.border, width: 1.5),
        borderRadius: BorderRadius.circular(10),
      ),
      padding: const EdgeInsets.symmetric(horizontal: 12),
      child: DropdownButtonHideUnderline(
        child: DropdownButton<T>(
          value: value,
          isExpanded: true,
          hint: Text(
            hint,
            style: const TextStyle(fontSize: 14, color: AppColors.textMuted),
          ),
          items: items.map((item) {
            return DropdownMenuItem(
              value: item,
              child: Text(
                itemLabel(item),
                style: const TextStyle(fontSize: 14),
              ),
            );
          }).toList(),
          onChanged: onChanged,
        ),
      ),
    );
  }
}
```

```
/// ■■■■■■■■■■■■
class _NoteInput extends StatefulWidget {
  final TextEditingController controller;
  final FocusNode? focusNode;

  const _NoteInput({
    required this.controller,
    this.focusNode,
  });

  @override
  State<_NoteInput> createState() => _NoteInputState();
}

class _NoteInputState extends State<_NoteInput> {
  int _charCount = 0;

  @override
  void initState() {
    super.initState();
    _charCount = widget.controller.text.length;
    widget.controller.addListener(_onTextChanged);
  }

  void _onTextChanged() {
    setState(() {
      _charCount = widget.controller.text.length;
    });
  }

  @override
  void dispose() {
    widget.controller.removeListener(_onTextChanged);
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Row(
          children: [
            Text('■', style: TextStyle(fontSize: 14)),
            SizedBox(width: 4),
            Text(
              '■■■■(■■)',
              style: TextStyle(
                fontSize: 13,
```

```
          fontWeight: FontWeight.w500,
          color: AppColors.textSecondary,
        ),
      ),
    ],
  ),
  const SizedBox(height: 6),
  Stack(
    children: [
      TextField(
        controller: widget.controller,
        focusNode: widget.focusNode,
        maxLength: 200,
        maxLines: 3,
        minLines: 3,
        style: const TextStyle(fontSize: 13),
        decoration: InputDecoration(
          hintText: '■■■■■(■■■■■■)...',
          hintStyle: TextStyle(
            fontSize: 13,
            color: Colors.grey.shade400,
          ),
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(10),
            borderSide: const BorderSide(
              color: AppColors.border,
              width: 1.5,
            ),
          ),
          enabledBorder: OutlineInputBorder(
            borderRadius: BorderRadius.circular(10),
            borderSide: const BorderSide(
              color: AppColors.border,
              width: 1.5,
            ),
          ),
          focusedBorder: OutlineInputBorder(
            borderRadius: BorderRadius.circular(10),
            borderSide: const BorderSide(
              color: AppColors.primary,
              width: 1.5,
            ),
          ),
          counterText: '',
          contentPadding: const EdgeInsets.fromLTRB(12, 10, 12, 24),
        ),
      ),
      Positioned(
        bottom: 8,
        right: 10,
```

```
              child: Text(
                '$_charCount/200',
                style: TextStyle(
                  fontSize: 11,
                  color: _charCount == 200
                      ? AppColors.primary
                      : const Color(0xFFADB5BD),
                ),
              ),
            ),
          ],
        ),
      ],
    );
  }
}

/// ■■■■■■■ Dialog ■■■
class _RecordCard extends StatelessWidget {
  final String bookTitle;
  final String? durationStr;
  final String? note;

  const _RecordCard({
    super.key,
    required this.bookTitle,
    this.durationStr,
    this.note,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: const Color(0xFFF8F9FA),
        borderRadius: BorderRadius.circular(10),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Text(
                '■ $bookTitle',
                style: const TextStyle(
                  fontSize: 14,
                  fontWeight: FontWeight.w600,
                  color: AppColors.textPrimary,
```

```
              ),
            ),
            const Spacer(),
            if (durationStr != null && durationStr!.isNotEmpty)
              Container(
                padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
                decoration: BoxDecoration(
                  color: const Color(0xFFFFF0F0),
                  borderRadius: BorderRadius.circular(10),
                ),
                child: Text(
                  durationStr!,
                  style: const TextStyle(
                    fontSize: 12,
                    fontWeight: FontWeight.w700,
                    color: AppColors.primary,
                  ),
                ),
              ),
          ],
        ),
        const SizedBox(height: 6),
        Text(
          note != null && note!.isNotEmpty
              ? note!
              : '(■■■■■)',
          style: TextStyle(
            fontSize: 13,
            color: note != null && note!.isNotEmpty
                ? const Color(0xFF495057)
                : const Color(0xFFADB5BD),
            fontStyle:
                note != null && note!.isNotEmpty ? FontStyle.normal : FontStyle.italic,
            height: 1.5,
          ),
        ),
      ],
    ),
  );
 }
}


import 'package:flutter/material.dart';
import '../theme/colors.dart';

/// ■■■■■V3.5 ■■■■■
/// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class CheckinOptionSheet extends StatelessWidget {
  final String dateStr;
```

```
final VoidCallback onNewCheckin;
final VoidCallback onViewDetail;

const CheckinOptionSheet({
  super.key,
  required this.dateStr,
  required this.onNewCheckin,
  required this.onViewDetail,
});

String _formatDate(String dateStr) {
  final parts = dateStr.split('-');
  if (parts.length != 3) return dateStr;
  return '${int.parse(parts[0])}■${int.parse(parts[1])}■${int.parse(parts[2])}■';
}

@override
Widget build(BuildContext context) {
  return Container(
    padding: const EdgeInsets.fromLTRB(24, 20, 24, 32),
    decoration: const BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
    ),
    child: Column(
      mainAxisSize: MainAxisSize.min,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // ■■
        Text(
          '■ ${_formatDate(dateStr)} · ■■■',
          style: const TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.w600,
            color: AppColors.textPrimary,
          ),
        ),
        const SizedBox(height: 20),

        // ■■■■■■■
        _OptionButton(
          icon: '■',
          label: '■■■■',
          onTap: onNewCheckin,
        ),
        const SizedBox(height: 8),

        // ■■■■■■■■
        _OptionButton(
          icon: '■',
```

```
            label: '■■■■■■■■',
            onTap: onViewDetail,
          ),
        ],
      ),
    );
  }
}

class _OptionButton extends StatelessWidget {
  final String icon;
  final String label;
  final VoidCallback onTap;

  const _OptionButton({
    super.key,
    required this.icon,
    required this.label,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: double.infinity,
      child: Material(
        color: const Color(0xFFF8F9FA),
        borderRadius: BorderRadius.circular(12),
        child: InkWell(
          onTap: onTap,
          borderRadius: BorderRadius.circular(12),
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
            child: Row(
              children: [
                Text(icon, style: const TextStyle(fontSize: 18)),
                const SizedBox(width: 12),
                Text(
                  label,
                  style: const TextStyle(
                    fontSize: 15,
                    fontWeight: FontWeight.w500,
                    color: AppColors.textPrimary,
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
```

```
    );
  }
}


import 'dart:math';
import 'package:flutter/material.dart';

/// ■■■■
class ConfettiPiece {
  final double left; // 0-100 (■■■)
  final double sizeW;
  final double sizeH;
  final Color color;
  final double delay; // ■
  final double duration; // ■
  final bool isCircle; // ■■■■■■
  late final double startRotation;
  late final double endRotation;

  ConfettiPiece({
    required this.left,
    required this.sizeW,
    required this.sizeH,
    required this.color,
    required this.delay,
    required this.duration,
    required this.isCircle,
    double? startRotation,
    double? endRotation,
  }) {
    this.startRotation = startRotation ?? Random().nextDouble() * 360;
    this.endRotation = endRotation ?? startRotation! + 360 + Random().nextDouble() * 360;
  }
}

/// ■■■■■■■
class ConfettiPainter extends CustomPainter {
  final List<ConfettiPiece> pieces;
  final double animationValue; // 0.0 ~ 1.0
  final double height;

  ConfettiPainter({
    required this.pieces,
    required this.animationValue,
    required this.height,
  });

  @override
  void paint(Canvas canvas, Size size) {
```

```
    final elapsed = animationValue * 3.5; // ■■■■■■ 3.5s

    for (final piece in pieces) {
      final localElapsed = elapsed - piece.delay;
      if (localElapsed < 0 || localElapsed > piece.duration) continue;

      final progress = localElapsed / piece.duration; // 0.0 ~ 1.0

      // Y ■■■■■■ (-20) ■■■■ (height + 40)
      final y = lerpDouble(-20, height + 40, progress)!;

      // X ■■■■■■■■■
      final x = (piece.left / 100) * size.width + sin(progress * 4 * pi) * 15;

      // ■■
      final rotation = piece.startRotation +
          (piece.endRotation - piece.startRotation) * progress;

      // ■■■■
      final opacity = progress < 0.85 ? 1.0 : (1.0 - (progress - 0.85) / 0.15);

      // ■■
      final scale = lerpDouble(1.0, 0.4, progress)!;

      canvas.save();
      canvas.translate(x, y);
      canvas.rotate(rotation * pi / 180);
      canvas.scale(scale);

      final paint = Paint()..color = piece.color.withValues(alpha: opacity);

      if (piece.isCircle) {
        canvas.drawCircle(Offset.zero, piece.sizeW / 2, paint);
      } else {
        canvas.drawRRect(
          RRect.fromRectAndRadius(
            Rect.fromCenter(
              center: Offset.zero,
              width: piece.sizeW,
              height: piece.sizeH,
            ),
            const Radius.circular(2),
          ),
          paint,
        );
      }

      canvas.restore();
    }
  }
```

```
  @override
  bool shouldRepaint(covariant ConfettiPainter oldDelegate) =>
      oldDelegate.animationValue != animationValue;

  static double? lerpDouble(double a, double b, double t) {
    return a + (b - a) * t.clamp(0.0, 1.0);
  }
}


import 'package:flutter/material.dart';
import '../theme/colors.dart';

/// ■■■■■■
class StarRating extends StatelessWidget {
  final int rating; // 0-5
  final ValueChanged<int>? onChanged;
  final double size;

  const StarRating({
    super.key,
    this.rating = 0,
    this.onChanged,
    this.size = 32,
  });

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: List.generate(5, (index) {
        final isFilled = index < rating;
        return GestureDetector(
          onTap: onChanged != null ? () => onChanged!(index + 1) : null,
          child: Padding(
            padding: const EdgeInsets.only(right: 4),
            child: Icon(
              isFilled ? Icons.star : Icons.star_border,
              size: size,
              color: isFilled ? AppColors.starActive : AppColors.border,
            ),
          ),
        );
      }),
    );
  }
}
```